COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S2
DATE PERFORMED:	09-18-2026	**DATE SUBMITTED:**	09-23-2026
NAME:	GUARIN, JAN MIKAELA C.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 9.1 Tree ADT

1. PROCEDURE OUTPUT

Table 9.1 - General Tree

Code Snippet

```
#ifndef GENERALTREE_H
#define GENERALTREE_H

#include <iostream>

template <typename T>
class GeneralTree
{
private:
    T key;
    GeneralTree<T> *firstChild;
    GeneralTree<T> *nextSibling;

public:
    // constructor
    GeneralTree(T rootObj);

    // inserting a child node, gets added after the existing ones and returns the new node
    GeneralTree<T> *insertChild(T newNode);

    GeneralTree<T> *getFirstChild();
    GeneralTree<T> *getNextSibling();

    T getRootVal();
    void setRootVal(T obj);

    bool isLeaf();

    // just to check the tree looks right
    void printTree(int depth = 0);
};

template <typename T>
GeneralTree<T>::GeneralTree(T rootObj)
{
    key = rootObj;
    firstChild = nullptr;
    nextSibling = nullptr;
}

template <typename T>
GeneralTree<T> *GeneralTree<T>::insertChild(T newNode)
{
    GeneralTree<T> *newChild = new GeneralTree<T>(newNode);

    if (firstChild == nullptr)
    {
        // no kids yet so this is the first one
        firstChild = newChild;
    }
    else
    {
        // otherwise go to the end of the sibling list and stick it there
        GeneralTree<T> *temp = firstChild;
        while (temp->nextSibling != nullptr)
        {
            temp = temp->nextSibling;
        }
        temp->nextSibling = newChild;
    }

    return newChild;
}

template <typename T>
GeneralTree<T> *GeneralTree<T>::getFirstChild()
{
    return firstChild;
}
```

```
}

template <typename T>
GeneralTree<T> *GeneralTree<T>::getNextSibling()
{
    return nextSibling;
}

template <typename T>
T GeneralTree<T>::getRootVal()
{
    return key;
}

template <typename T>
void GeneralTree<T>::setRootVal(T obj)
{
    key = obj;
}

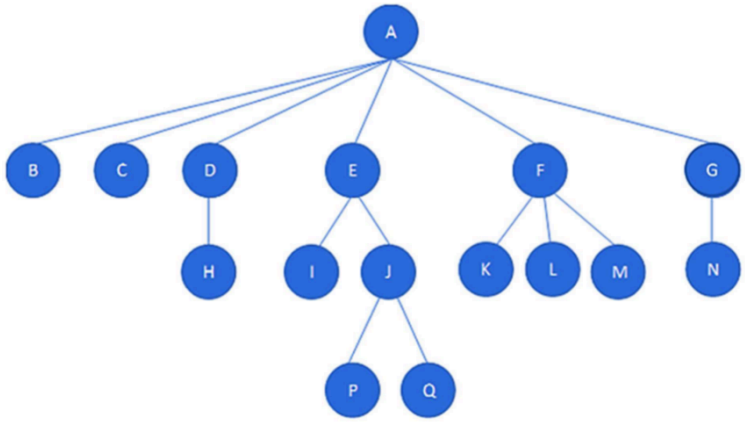
template <typename T>
bool GeneralTree<T>::isLeaf()
{
    return firstChild == nullptr;
}

template <typename T>
void GeneralTree<T>::printTree(int depth)
{
    for (int i = 0; i < depth; i++)
    {
        std::cout << "  ";
    }
    std::cout << key << std::endl;

    if (firstChild != nullptr)
    {
        firstChild->printTree(depth + 1); // go down
    }
    if (nextSibling != nullptr)
    {
        nextSibling->printTree(depth); // next one over
    }
}

#endif
```

Tree Diagram



Observation

The code successfully creates and displays the general tree structure by connecting each parent node to its corresponding child nodes. The isLeaf() function checks whether a node has no child, while the printTree() function displays each node with indentation based on its depth in the tree. The diagram shows A as the root node, with several children and deeper levels such as D–H, E–I/J, J–P/Q, F–K/L/M, and G–N. The output demonstrates how the code uses recursion to move down through the child nodes and then moves to the next sibling, allowing the entire tree to be displayed in its proper hierarchical structure.

Table 9.2

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	2
H	0	2
I	0	2
J	1	2
K	0	2
L	0	2
M	0	2
N	0	2
O	0	3
P	0	3
Q	0	3

Table 9.3

Pre-order	ABCDHEIJPQFKLMGN
In-order	BACHDIEPJQKFLMNG
Post-Order	BCHDIPQJEKLMFNGA

Table 9.4

Pre-order Snippet	
Pre-order Output	
In-order Snippet	
In-order Snippet	

```
// Pre-order
template <typename T>
void preOrder(GeneralTree<T> *node)
{
    if (node == nullptr)
        return;

    cout << node->getRootVal() << " ";

    GeneralTree<T> *child = node->getFirstChild();

    while (child != nullptr)
    {
        preOrder(child);
        child = child->getNextSibling();
    }
```

```
Pre-order: A B C D H E I J P Q F K L M G N
```

```
// In-order
template <typename T>
void inOrder(GeneralTree<T> *node)
{
    if (node == nullptr)
        return;

    GeneralTree<T> *firstChild = node->getFirstChild();

    if (firstChild != nullptr)
    {
        inOrder(firstChild);
    }

    cout << node->getRootVal() << " ";

    if (firstChild != nullptr)
    {
        GeneralTree<T> *nextChild = firstChild->getNextSibling();

        while (nextChild != nullptr)
        {
            inOrder(nextChild);
            nextChild = nextChild->getNextSibling();
        }
    }
}
```

```
In-order: B A C H D I E P J Q K F L M N G
```

Post-order Snippet	
Post-order Output	
Observation	In creating these traversal functions, I observed that all three use the same recursion: stop at nullptr, start at getFirstChild(), and move through the children with getNextSibling(). The only difference is where I placed the cout. In pre-order, I print the node before its children, so parents come first (A B C D H E I J P Q F K L M G N). In in-order, I visit the first child, print the node, then visit the remaining siblings (B A C H D I E P J Q K F L M N G). In post-order, I print the node after all its children, so the root A comes last (B C H D I P Q J E K L M F N G A). This shows that moving one print statement changes the visiting order without changing the recursion, and each function visits every node once.

Table 9.5

Code Snippet

```
template <typename T>
void GeneralTree<T>::findData(std::string choice, T searchKey)
{
    bool found = false;

    // ---------------- PRE-ORDER ----------------

    if (choice == "PRE")
    {
        if (key == searchKey)
        {
            std::cout << searchKey
                      << " was found!" << std::endl;

            return;
        }

        GeneralTree<T> *child = firstChild;

        while (child != nullptr)
        {
            child->findData(choice, searchKey);

            child = child->nextSibling;
        }
    }

    // ---------------- POST-ORDER ----------------

    else if (choice == "POST")
    {
        GeneralTree<T> *child = firstChild;

        while (child != nullptr)
        {
            child->findData(choice, searchKey);

            child = child->nextSibling;
        }

        if (key == searchKey)
        {
            std::cout << searchKey
                      << " was found!" << std::endl;
        }
    }
```

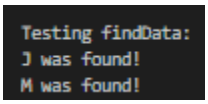
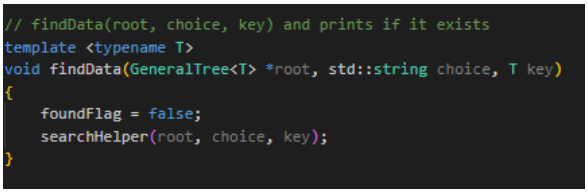
	
Output	
Observations	In creating this findData function, I observed that the "IN" branch follows the in-order pattern. It first calls findData on the first child, then checks if the current node's key matches the searchKey and prints "was found!", and finally moves to nextSibling to search the remaining children the same way. Since the search is recursive, every node is checked exactly once. The output shows that J and M were both found, which means both keys exist in the tree. This also shows that the order of the printed results follows the traversal order, and that the check is done in the middle, after the first child and before the remaining siblings.

Table 9.6

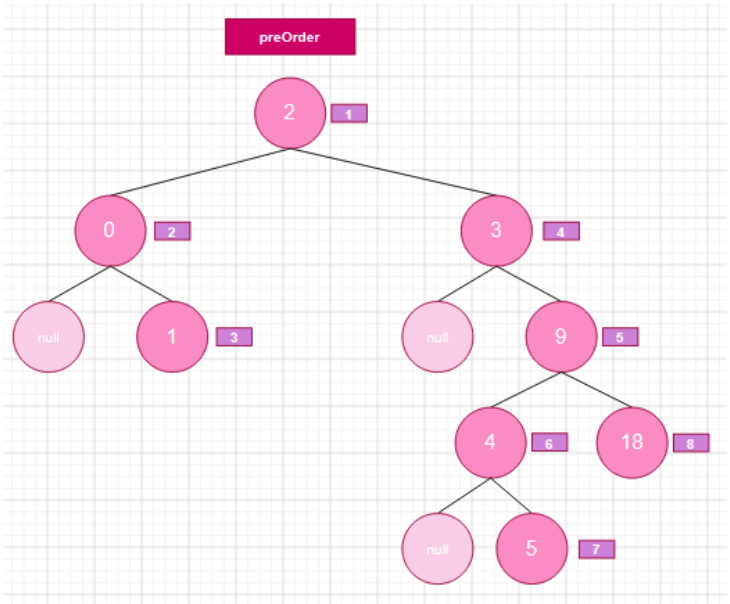
Code Snippet	

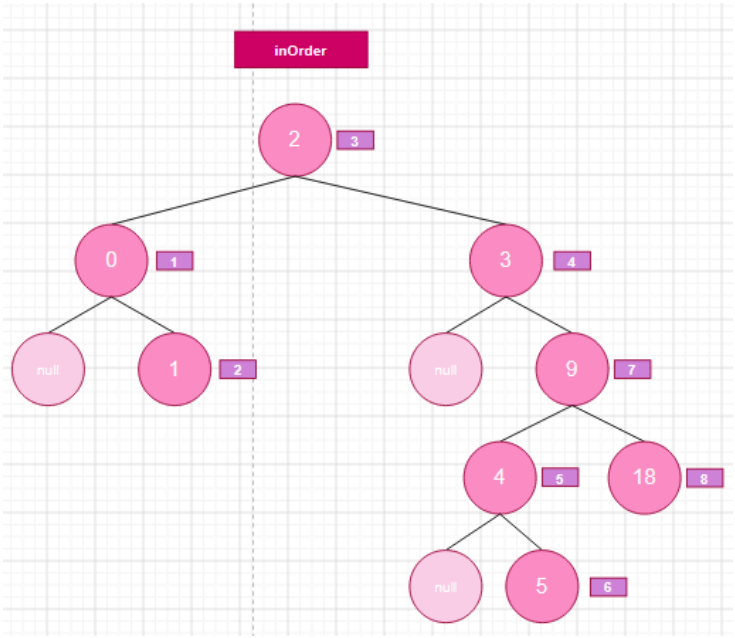
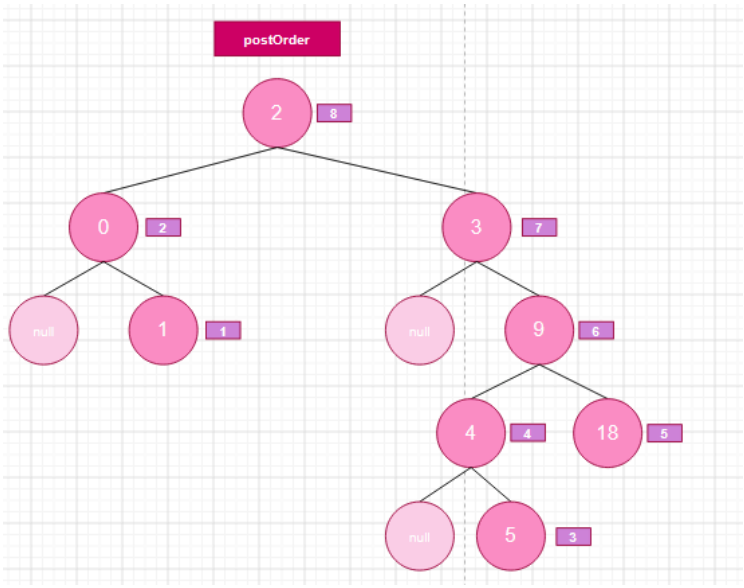
Output	`Searching for 0:` `0 was found!`
Observations	In creating this function, I observed that findData only sets foundFlag to false and then calls searchHelper to do the actual searching. The output shows that 0 was found, which means 0 exists in the tree.

REFERENCES:

https://www.tutorialspoint.com/article/difference-between-general-tree-and-binary-tree
https://www.eecs.yorku.ca/~wangcw/teaching/lectures/2022/W/EECS2011/slides/05a-Tree-ADT.pdf

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Traversal	Diagram
Pre-order	

In-order	
Post-order	

Code Snippet

```cpp
// pre order
template <typename T>
void BinaryTree<T>::preOrder()
{
    std::cout << key << " ";

    if (leftChild != nullptr)
    {
        leftChild->preOrder();
    }

    if (rightChild != nullptr)
    {
        rightChild->preOrder();
    }
}
```

Output

```
preOrder: 2 0 1 3 9 4 5 18
```

Code Snippet

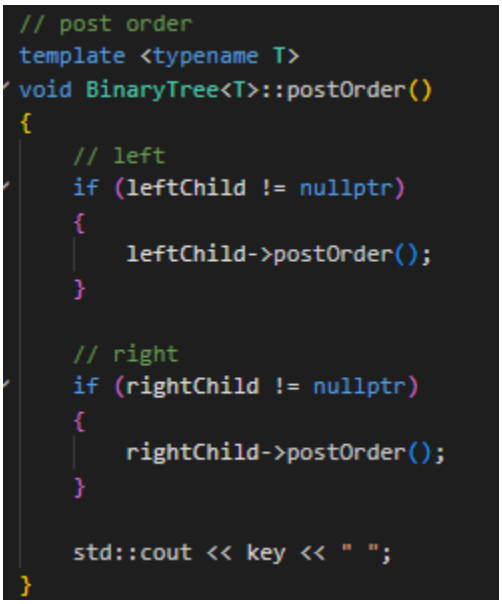
```cpp
// in order
template <typename T>
void BinaryTree<T>::inOrder()
{
    // left
    if (leftChild != nullptr)
    {
        leftChild->inOrder();
    }

    std::cout << key << " ";

    // right
    if (rightChild != nullptr)
    {
        rightChild->inOrder();
    }
}
```

Output

```
inOrder: 0 1 2 3 4 5 9 18
```

Code Snippet	
Output	
Observations	In creating these functions, I observed that postOrder visits the left child, then the right child, and prints the node's key last, so children always print before their parent, giving 1 0 5 4 18 9 3 2 with the root 2 last. The preOrder function prints the key first, then visits the left and right children, so parents print before their children, giving 2 0 1 3 9 4 5 18 with the root 2 first. The inOrder function visits the left child, prints the key, and then visits the right child, and since this is a binary search tree, the values print in sorted order: 0 1 2 3 4 5 9 18.

```
// post order
template <typename T>
void BinaryTree<T>::postOrder()
{
    // left
    if (leftChild != nullptr)
    {
        leftChild->postOrder();
    }

    // right
    if (rightChild != nullptr)
    {
        rightChild->postOrder();
    }

    std::cout << key << " ";
}
```

```
postOrder: 1 0 5 4 18 9 3 2
```

CONCLUSION:

At first, this activity seemed challenging and confusing because the recursive calls were hard to follow, and I could not easily tell why the same code produced different outputs. As I traced each function step by step, I realized that the recursion is the same in all three and that only the position of the cout changes the order. I also learned how the findData function searches for a key in the tree by using a helper function. By the end, I understood how preOrder, inOrder, and postOrder work, and that the traversal order depends on when the node is visited. This activity improved my understanding of trees and recursion.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.